

Routing3D 개발 보고서

플랜트 배관 3D 직교 자동 라우팅 엔진 · 전체 + 단계별(Phase 1~3) · 흐름도 / 알고리즘
/ 주요 함수·변수 / 결과

단위 mm · 기본 셀 50mm · 작성일 2026-05-29 · 레퍼런스: python_experiments/ · C++ 엔진:
cpp/

0. 문서 개요

본 보고서는 Routing3D 엔진의 전체 개발 내용을 단계(step)별로 정리한 개발보고서다. 각 단계는 '무엇을·왜·어떻게'를 흐름도와 알고리즘으로 설명하고, 구현의 자료구조·주요 함수·주요 변수, 그리고 측정된 결과와 실행명령어를 포함한다.

구성은 프로젝트 전체 개요(1 장) → Phase 1 Python 알고리즘 실험(2 장) → Phase 2 인터페이스 동결(3 장) → Phase 3 C++ 엔진 + pybind11(4 장) → C++↔C# 인터럽·HelixToolkit 뷰어(5 장) → 전체 결과·향후 과제(6 장) 순이다.

0.1 목차

- 1. 프로젝트 전체 개요 — 목적 / 3 단계 전략 / 아키텍처 / 개념 / 환경 / 불변식
- 2. Phase 1 — 점유맵(2.1) · 직교 A*(2.2) · 비용함수(2.3) · 다중배관(2.4) · scene.txt I/O(2.5) · 성능 프로파일(2.6) · 회귀 골든셋(2.7)
- 3. Phase 2 — 알고리즘 명세(3.1) · scene.txt v1 규격(3.2) · 회귀 골든셋(3.3) · 성능 목표(3.4)
- 4. Phase 3 — 빌드 체계(4.1) · 점유 백엔드(4.2) · 직교 A*(4.3) · 비용함수(4.4) · 다중배관+rip-up(4.5) · scene.txt I/O(4.6) · 계층 corridor(4.7) · FCL 충돌(4.8) · pybind11(4.9) · 실행 CLI(4.10)
- 5. C++↔C# 인터럽 — C ABI DLL(5.1) · HelixToolkit 뷰어(5.2) · 실테이터 교차검증(5.3)
- 6. 전체 결과 요약 및 향후 과제

1. 프로젝트 전체 개요

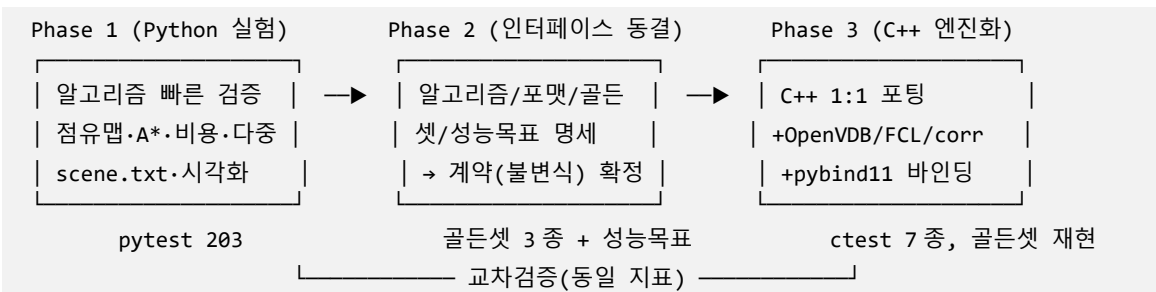
1.1 목적 · 배경

Routing3D 는 플랜트(반도체 FAB 등) 공간에서 여러 배관(파이프)의 경로를 3 차원 직교(맨해튼) 방식으로 자동 설계하는 엔진이다. 장애물(구조물·장비)을 피하고, 배관끼리 충돌하지 않으며, 회전(엘보)을 최소화하는 경로를 찾는다. 모든 좌표·치수의 단위는 밀리미터(mm)다.

- 신규 개발: 기존 C++ 프로토타입·뷰어 없음(인접 SpaceAI C# 구현은 개념·DB 참조만, 직접 포팅 안 함).
- 입력: 장애물 AABB 목록 + 라우팅 작업(start→end PoC 페어) + 비용 파라미터.
- 출력: 각 배관의 직교 경로(셀 시퀀스) + 지표(길이/회전/탐색량/충돌수).

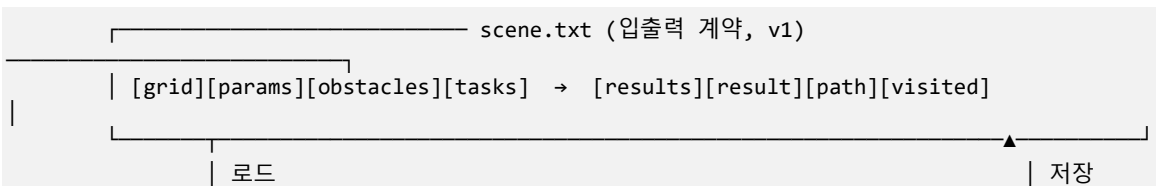
1.2 3 단계 개발 전략

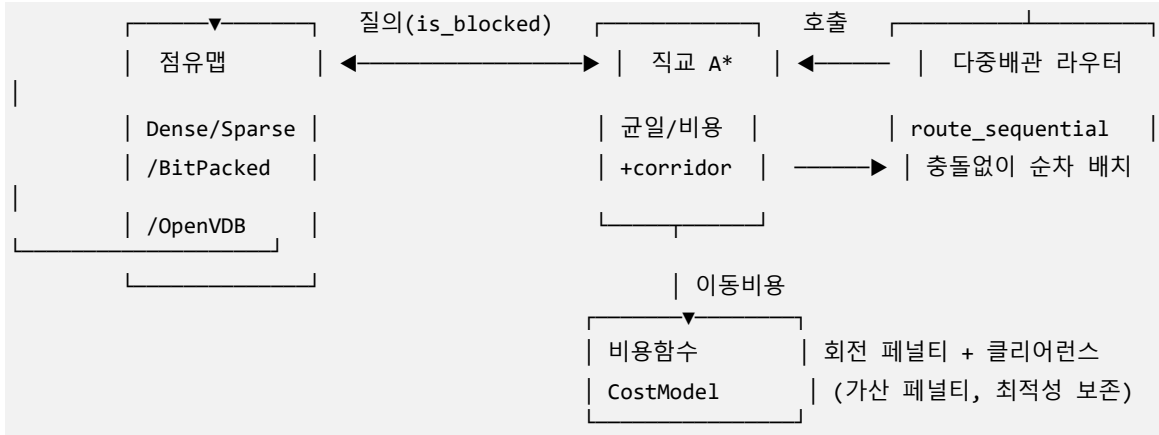
알고리즘을 Python 으로 먼저 검증(Phase 1)하고, 인터페이스·포맷·골든셋을 동결(Phase 2)한 뒤, 동일 동작을 C++ 로 1:1 포팅(Phase 3)한다. 단계마다 Python 레퍼런스와 교차검증하여 회귀를 막는다.



1.3 전체 아키텍처

점유맵 위에서 A* 가 경로를 찾고, 비용함수가 회전·클리어런스를 반영하며, 다중배관 라우터가 여러 배관을 충돌 없이 순차 배치한다. scene.txt 가 입출력 계약을 담당한다. 같은 구조를 Python(실험)과 C++(엔진)이 공유한다.





1.4 핵심 개념 · 용어

용어	의미
셀(cell)	정수 인덱스 (i,j,k). 공간을 cell_mm 정육면체로 나눈 격자 단위.
월드(world)	실제 좌표 (x,y,z) mm. to_world/to_cell 로 셀과 변환.
점유맵	각 셀이 장애물로 막혔는지(True/False) 관리. A* 의 통행 가능 질의 대상.
직교 A*	6 방향(±X,±Y,±Z)만 이동하는 최단경로 탐색(대각선 금지).
클리어런스	장애물에서 떨어진 정도. 가까울수록 비용 가산(벽 회피 유도).
다중배관 순차	이미 깔린 배관을 점유로 추가하며 한 개씩 라우팅(충돌 0).
계층 corridor	coarse 격자로 대략 경로 → 그 주변만 fine 정밀 탐색(초대형 대응).

1.5 개발 환경 · 빌드 · 실행

항목	내용
Python	3.13, 루트 .venv (editable 설치: pip install -e "python_experiments[viz]")
C++	C++20, MSVC VS2022 Pro 17.14, CMake + Visual Studio 17 2022 생성기
필수 플래그	MSVC /utf-8 (한글 주석 CP949 오해 방지)
무거운 의존성	vcpkg(D:/vcpkg): OpenVDB 12.0.1+TBB, FCL 0.7.0, pybind11 3.0.4(pip)
단위 테스트	Python: pytest 203 통과 / C++: ctest 7 종(golden-scene_io-occupancy-corridor-vdb-fcl-bindings)

```
# Python 테스트
.\.venv\Scripts\python.exe -m pytest python_experiments

# C++ 빌드 + 테스트 (코어; OpenVDB/FCL/bindings 는 옵션 ON 시)
cmake -S cpp -B cpp/build -G "Visual Studio 17 2022" -A x64
cmake --build cpp/build --config Release
ctest --test-dir cpp/build -C Release --output-on-failure

# 엔진 실행(CLI)
.\run.ps1                # 내장 데모(골든 03)
.\run.ps1 route --in scene.txt --out routed.scene.txt --mode multi
```

1.6 핵심 불변식(계약) — Phase 1~3 공통

Python 과 C++ 가 동일 동작임을 보증하는 계약이다. 단위 테스트로 강제하며, C++ 포팅 시에도 자료구조가 바뀌어도(dict→배열, heapq→해시) 동일하게 보존된다.

ID	불변식
G1	격자 범위 밖 셀은 항상 점유로 간주(A* 가 격자 밖으로 못 나감).
O1	점유맵 백엔드(Dense/BitPacked/Sparse/VDB)는 동일 질의에 동일 결과.
A1/C1	비용 가산항 $\geq 0 \rightarrow$ 맨해튼 휴리스틱 admissible & consistent(최적성 보존).
A2/W1	동일 입력 $\rightarrow$ 동일 경로. tie-break=(f, 삽입순서), 이웃 순서 고정.
P1	경로의 모든 셀은 비점유, 연속 셀은 6-이웃(직교 연결).
M1	다중배관 성공 경로는 쌍별로 셀 비공유(충돌 0).
M2	순차 라우팅은 입력 점유맵을 변경하지 않음(사본 사용).
F2/F3/F4	scene.txt write $\rightarrow$ read $\rightarrow$ write 바이트 동일, WN(None) $\neq$ ""(빈), 실수 repr 보존.

2. Phase 1 — Python 알고리즘 실험

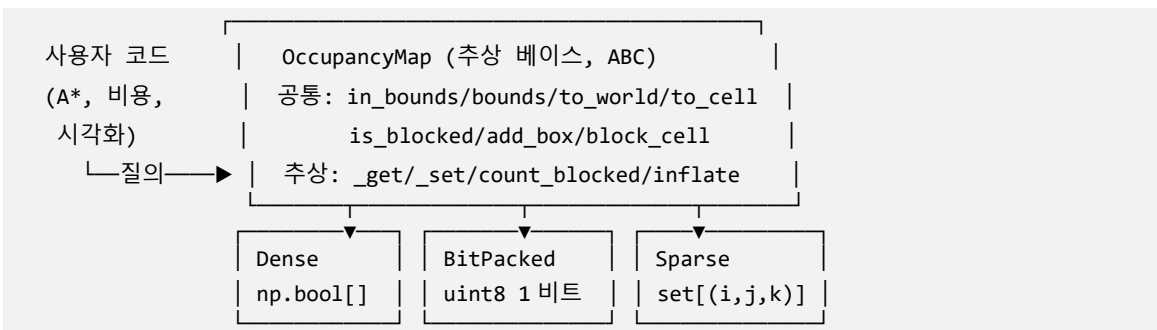
라우팅 핵심 알고리즘을 Python 으로 구현·검증한다(모듈 routing3d_py). 빠른 반복으로 동작을 확정하고, 이후 C++ 포팅의 기준(레퍼런스)이 된다. pytest 203 건 통과.

2.1 점유맵 (occupancy.py)

개요

플랜트 공간을 cell_mm(기본 50mm) 정육면체 셀의 3D 격자로 표현하고, 각 셀의 점유 여부(True/False)를 관리한다. A* 는 이 점유맵에 '이 셀로 지나갈 수 있는가?'를 질의한다. 같은 질의 인터페이스(OccupancyMap 추상클래스) 뒤에 저장 방식이 다른 백엔드 3 종을 둔다.

구조 흐름도



알고리즘 (좌표/복셀화)

- $\text{to_world}(\text{cell}) = \text{origin} + (\text{cell} + 0.5) \times \text{cell_mm} \rightarrow \text{셀 중심 월드좌표(mm)}$.
- $\text{to_cell}(\text{world}) = \text{floor}((\text{world} - \text{origin}) / \text{cell_mm}) \rightarrow \text{포함 셀 인덱스}$.
- $\text{add_box}(\text{AABB})$: $\text{lo}=\text{floor}$, $\text{hi}=\text{ceil}$ 로 셀 범위 산출 $\rightarrow [0, \text{shape})$ 클리핑 $\rightarrow$ 채우고 신규 점유 셀 수 반환.
- $\text{inflate}(r, \text{connectivity})$: r 회 OR-시프트 팽창(6=면 인접, 26=면+모서리+꼭짓점). 하드 클리어런스.

자료구조 / 주요 함수

이름	역할
AABB(frozen dataclass)	장애물 직육면체(lo<hi). 필드 lo/hi=(x,y,z) mm.
OccupancyMap(ABC)	질의 인터페이스 + 공통 좌표/복셀화 로직.
Dense/BitPacked/SparseOccupancyMap	각각 np.bool 배열 / uint8 비트팩 / 좌표 set

	백엔드.
is_blocked(cell)	점유 또는 격자 밖이면 True(A* 질의 핵심).
add_box / add_boxes	AABB 복셀화. 신규 점유 셀 수 반환.
inflate / copy / count_blocked	팽창 새 맵 / 독립 복사(다중배관용) / 점유 셀 수.

주요 변수

이름	설명
shape=(nx,ny,nz)	각 축 셀 개수.
origin / cell_mm	격자 원점(mm) / 셀 한 변(mm, 기본 50).
NEIGHBORS_6 / _26	면 인접 6 방향 / 면+모서리+꼭짓점 26 방향(순서 고정).

결과

백엔드	셀당 메모리	적합 상황	실측(25.6% 점유, 50mm)
Dense	1 byte	작은 ROI, 최속 질의(기본)	625 KB
BitPacked	1 bit	같은 메모리로 ~8 배 큰 ROI	78 KB
Sparse	set 엔트리	점유가 희박할 때만	23.6 MB

주의: 바닥·기둥처럼 점유가 뿔뿔하면 Sparse 가 오히려 Dense 의 ~37 배까지 커진다.

균일한 큰 덩어리의 진짜 압축(OpenVDB)은 Phase 3 에서 도입한다.

실행명령어

```
.\.venv\Scripts\python.exe -m pytest python_experiments/tests/test_occupancy.py -v
```

2.2 직교 A* (astar.py)

개요

점유맵 위에서 시작 셀 → 목표 셀의 최단 직교 경로를 A* 로 찾는다. 6 방향 직교만 허용, 휴리스틱은 맨해튼 거리 × cell_mm (직교 격자에서 admissible & consistent).

탐색 흐름도

```
open(우선순위 큐): f = g + h 작은 것부터 pop
  ▼ pop current
current == goal ? —예—▶ came_from 역추적 → 경로 복원·반환
  | 아니오 (6-이웃 nb 마다)
is_blocked(nb)? —예—▶ 건너뛴(장애물/격자 밖)
  | 아니오
```

```
g_new = g[cur] + step_cost, g_new < g[nb] ? —예—▶ 갱신.push
    ↳ open 소진 → 경로 없음(None)
```

알고리즘 핵심

- $f(n) = g(n) + h(n)$. g =시작부터 누적 실비용, h =목표까지 추정.
- admissibility: 한 칸= cell_mm , h =맨해튼 $\times\text{cell_mm} \rightarrow$ 과대평가 없음 $\rightarrow$ 최적 경로 보장.
- tie-break: 힙 항목에 단조증가 counter $\rightarrow f$ 동률 시 안정 정렬(결정성 A2).
- closed 집합: 확정 셀의 낡은 중복 힙 항목 무시.

자료구조 / AStarResult

필드	설명
success / path	성공 여부 / 셀 리스트 [start..goal](실패 시 None).
length_mm	기하 길이 = (셀 수 - 1) $\times$ cell_mm.
turns / expanded_nodes	방향 전환 수 / 확장 노드 수(탐색량 지표).
cost_mm / elapsed_ms	총 비용(균일이면 length 와 동일) / 소요(ms).
visited	확장 셀 전체(collect_visited=True, 시각화용).

주요 함수 / 변수

이름	역할
astar(occ, start, goal, *, step_cost, ...)	균일 비용 직교 A*. 상태=셀.
astar_weighted(occ, start, goal, params)	비용함수 A*. 상태=(셀,진입방향)(Step 1.3).
manhattan(a,b) / count_turns(path)	맨해튼 거리 / 회전 수.
step_cost(=cell_mm) / max_expansions	한 칸 비용 / 확장 상한(폭주 방지).

결과

- 골든 01(빈 20^3): length 2850mm, turns 2, expanded 22,856.
- 골든 02(80^3 벽 우회): length 3950mm, turns 2, expanded 9,036.
- 동일 입력 $\rightarrow$ 동일 경로(결정성). 실패 조건: start/goal 점유·격자 밖, 경로 없음, 확장 상한 초과.

실행명령어

```
.\.venv\Scripts\python.exe -m routing3d_py.astar ^
--region 195000 8000 14000 205000 12000 16000 --cell-mm 50 ^
--start 196000 9000 15600 --goal 204000 11000 15600 ^
--screenshot python_experiments/out/route.png
```


2.3 비용함수 (cost.py)

개요

기본 이동비용(셀=cell_mm) 위에 회전 페널티(엘보 최소화), 클리어런스 페널티(벽 회피), 단(段) 분리 페널티(z 레벨 유도)를 더한다.

핵심 설계 결정 — 왜 '보너스'가 아니라 '페널티'인가

비용을 감산(보너스)하면 한 칸 비용이 cell_mm 보다 작아져 맨해튼 휴리스틱이 실제 비용을 과대평가 → A* 최적성(admissibility)이 깨진다. 그래서 동일 목적을 '장애물 근접 시 가산하는 페널티'로 구현한다. 모든 가산항이 ≥ 0 이므로 한 칸 이동 $\geq$ cell_mm 보장 → 최적성 보존(계약 C1).

흐름도 / 알고리즘

```
RouteParams(가중치) + OccupancyMap
    ▼ CostModel(occ, params) # 생성 시 클리어런스 맵 1 회 사전계산
    ▼ move_cost(to, prev_off, move_off)
    = cell_mm
    + (w_turn          if 방향 바뀜)
    + 클리어런스 페널티(to) # 장애물에 가까울수록 큼
    + 단 분리 페널티(to.z)
```

- clearance_map: bounded distance transform. 장애물=0, 1 단계씩 팽창하며 거리 기록(상한 R).
- cell_penalty(c): $d < \text{clearance_radius}$ 면 $w_clear \times (\text{clearance_radius} - d)$ 가산 + w_tier 가산.
- astar_weighted 연동: 회전 판정 때문에 상태를 (셀, 진입방향)으로 확장(진입방향=NEIGHBORS_6 인덱스, 시작 -1).

주요 함수 / 변수 (RouteParams)

필드	기본값	설명
cell_mm	50.0	셀 1 칸 기본 비용(mm).
w_turn	500.0	회전 1 회 가산(mm). 클수록 회전 강하게 회피.
w_clear	10.0	클리어런스 페널티 계수(mm/셀). 0=비활성.
clearance_radius	2	페널티 적용 최대 근접 거리(셀).
clearance_connectivity	6	거리 측정 이웃(6/26).

w_tier	{}	단 분리 가중치 {z 셀: 가산 mm}.
--------	----	------------------------

결과 / 실행명령어

회전·클리어런스가 반영된 경로가 벽에서 떨어져 매끄럽게 이어짐을 시각화로 확인.

test_cost.py 통과.

```
.\.venv\Scripts\python.exe -m pytest python_experiments/tests/test_cost.py -v
```

2.4 다중 배관 순차 라우팅 (multi_route.py)

개요

여러 배관(start→end 작업)을 한 개씩 차례로 라우팅한다. 핵심은 이미 깔린 배관을 다음 배관의 장애물로 추가하여 배관끼리 같은 셀을 점유하지 않게(충돌 0) 만드는 것(greedy sequential 베이스라인).

순차 라우팅 흐름도

- 1) 우선순위로 작업 순서 결정(기본: 긴 것 먼저)
- 2) 장애물 점유맵의 작업용 사본 생성(원본 보존: `occ.copy()`)
- 3) 작업을 순서대로:
 - a) start/end 가 점유면 가까운 빈 셀로 스냅(`snap_to_free`)
 - b) 비용함수 $A^*(\text{astar_weighted})$ 로 경로 탐색
 - c) 성공 → 경로 셀(+`pipe_radius` 팽창)을 점유로 추가 → 이후 배관이 회피
 - d) 실패 → 기록만(다음 배관에 영향 없음)
- 4) 성공률 / 총 길이 / 실패 수 측정

우선순위 규칙 / 주요 함수

이름	설명
priority=longest	시작-끝 맨해튼 거리 긴 배관 먼저(기본, 어려운 것 먼저).
shortest / utility / original	짧은 것 먼저 / 유틸 그룹 후 거리 / 입력 순서.
route_sequential(...)	순차 라우팅 본체. 사본에 배관 누적하며 충돌 없이 배치.
order_tasks / _mark_pipe / _snap	정렬(원본 불변) / 경로 점유표시 / 점유 시 빈 셀 스냅.

주요 변수 / 결과(MultiRouteResult)

- `pipe_radius`(기본 0): 깔린 배관 팽창 반경(셀). `snap_to_free`(기본 2): 빈 셀 탐색 반경.
- 지표: `success_count` / `fail_count` / `success_rate` / `total_length_mm` / `by_utility()`.

- 실측 project6: 203/208 성공(98%), 충돌 0, 총 3,677,400mm. 실패 5 건은 혼잡 출발부 경합(Phase 3 rip-up/CBS 대상).

실행명령어

```
.\.venv\Scripts\python.exe -m routing3d_py.scene --project 6 --multi ^
--priority longest --cell-mm 100 --screenshot python_experiments/out/multi.png
```

2.5 scene.txt 입출력 (scene_io.py, Step 1.5)

개요

라우팅 실행의 입력(격자·파라미터·장애물·작업)과 출력(경로·방문·지표)을 사람이 읽는 텍스트 한 파일로 직렬화한다. write→read→write 가 바이트 단위로 동일(무손실)하며, Phase 2/3 의 Python↔C++ 공유 계약이 된다.

포맷 / 무손실 핵심

- 섹션 헤더 [name] + TAB 구분 행. 한 줄=한 레코드(또는 키-값). 파서는 단순 상태기계.
- 실수: Python repr(float) 로 기록(재파싱 시 동일 float, F4).
- null: 토큰 WN — 빈 문자열("")과 구분(F3).
- 3 개 레이어: 점유([obstacles]) · 경로([path]) · 방문([visited]).

주요 함수 / 결과

함수	역할
dump_scene / load_scene	SceneDoc ↔ 문자열 직렬화/역직렬화.
read_scene / write_scene	파일 입출력.
occupancy_from_doc	문서의 격자·장애물로 점유맵 재구성.

결과: project6(장애물 983·작업 208) 왕복 OK. 이름 필드의 공백·특수문자도 TAB·WN 규칙으로 무손실.

실행명령어

```
.\.venv\Scripts\python.exe -m routing3d_py.scene_io --project 6 --cell-mm 100 --multi --out
scene.txt
.\.venv\Scripts\python.exe -m routing3d_py.scene_io --in scene.txt --roundtrip
```

2.6 성능 프로파일 · Numba 미채택 (Step 1.6)

프로파일 결과 A* 루프가 시간의 ~100% 를 차지하지만, 그 비용은 순수 파이썬 인터프리터 오버헤드였다. Numba 가속(평면배열 포크)은 코드 이원화 비용 대비 이득이 제한적이라 미채택하고, 진짜 가속은 Phase 3 C++ 로 이연했다.

측정	값
직교 A*(균일) 120 ³ , 772K 확장	~2.6 s
비용함수 A* 120 ³ , 1.33M 확장	~28 s
Python 처리량	≈ 5 만 확장/초(인터프리터 지배)

2.7 회귀 시나리오 골든셋 3 종 (Step 1.7)

대표 시나리오를 입력 → 기대지표(허용범위)로 고정한다. A* 가 결정적이므로 길이/회전/충돌/총길이를 회귀를 검출한다. Phase 3 C++ 엔진은 동일 시나리오에서 이 지표를 재현해야 한다(합격 기준).

시나리오	격자 / 장애물	기대지표
01_single_empty	20 ³ , 장애물 없음	length 2850mm, turns 2, expanded ≤ 30000(측정 22,856)
02_single_obstacle	80 ³ , 벽 우회	length 3950mm(직선×1.145), turns 2, detour ≤ 1.2, expanded ≤ 12000(9,036)
03_multi_tier	120×120×60, 바닥 슬래브, 5 배관	success 5/5, collisions 0, total 28050mm

tests/scenarios/<name>/{input.json, expected_metrics.json}, 실행기 scenario_runner.py, 하니스 test_scenarios.py. baseline_params.json 확정.

```

.\.venv\Scripts\python.exe python_experiments/tests/scenario_runner.py
.\.venv\Scripts\python.exe -m pytest python_experiments/tests/test_scenarios.py -v

```

2.8 Phase 1 결과 요약

- Step 1.1~1.7 + scene.txt I/O + 시각화 + 회귀 시나리오 3 종 + baseline_params.json 완료(2026-05-28).
- pytest 203 통과(+DB 의존 6 건 deselect). origin/main 푸시 완료.
- 실데이터 project6: 208 배관 중 203 성공(98%), 충돌 0, 총 3,677,400mm.
- 상세 설계 문서 docs/spec/step1_1~1_4.{docx,pdf} 작성(생성기 _gen_spec_docs.py).

3. Phase 2 — 인터페이스 동결

Phase 1 에서 검증된 동작을 명세로 고정한다. Phase 3 C++ 구현이 모호함 없이 따를 수 있도록 알고리즘 의사코드·데이터 포맷·골든셋·성능목표를 문서화하고 계약(불변식)을 확정한다. 산출물: docs/spec/{algorithm_spec, scene_format_spec, regression_set, performance_targets, freeze_signoff}.md.

3.1 알고리즘 명세 (algorithm_spec.md)

좌표·격자 규약, 점유맵 질의 인터페이스, 균일 A*, 비용함수 A*, 다중배관 순차를 의사코드로 1:1 명세한다. 레퍼런스(aster.py/cost.py/multi_route.py/occupancy.py)와 정확히 대응.

- 균일 A*: 상태=셀, $h = \text{manhattan} \times \text{cell_mm}$, tie-break=(f, counter), 이웃 순서 고정.
- 비용함수 A*: 상태=(셀, 진입방향 dir). 시작 dir=-1. CostModel 이 클리어런스 1 회 사전계산.
- 다중배관: work=occ.copy() → order_tasks → 각 작업 snap→astar_weighted→성공 시 mark_pipe.
- 불변식 G1/O1/A1·C1/A2·W1/P1/M1/M2 를 표로 동결(\$6).

```
NEIGHBORS_6 = [(+1,0,0),(-1,0,0),(0,+1,0),(0,-1,0),(0,0,+1),(0,0,-1)] # 순서 고정
baseline RouteParams = {cell_mm:50, w_turn:500, w_clear:10,
                        clearance_radius:2, clearance_connectivity:6, w_tier:{}}
```

3.2 scene.txt v1 규격 (scene_format_spec.md)

중간 데이터 포맷의 정식 규격. v1 동결. UTF-8/LF/TAB 구분, # 주석, @format·@version 헤더, [name] 섹션. 무손실 핵심=실수 repr 보존 + \N(None)≠""(빈). 파일 구조 순서 고정.

```
[grid]    cell_mm/origin/shape      (필수)
[params]  RouteParams 키-값         (필수)
[obstacles] count=N → 10 필드 행    (minx..maxz, ost_type, name, object_id, ddworks_type)
[tasks]   count=M → 11 필드 행      (sx..gz, utility, utility_group, start/end_name, end_guid)
[results] count=K → [result]/[path]/[visited] (task=idx)
```

불변식 F1~F6(cell_mm 필수, 왕복 무손실, null 구분, repr 보존, count 일치, task idx 대응). 버전 정책: 비호환 변경은 @version 2 로 올리고 골든셋 재생성.

3.3 회귀 골든셋 (regression_set.md)

골든 01/02/03 의 입력과 기대지표(정확 일치 항목 vs 근사/상한 항목)를 동결. Phase 3 합격 기준= C++ 가 동일 입력으로 이 지표를 허용범위 내 재현 + scene.txt v1 무손실 + 불변식 단위테스트 통과.

3.4 성능 목표 (performance_targets.md)

사용자 확정 목표를 합격 기준으로 고정한다. 8,000m 스케일은 Dense 전면 그리드가 불가($\sim 4 \times 10^{15}$ 셀, 페타바이트)하므로 OpenVDB 희소 복셀 + 계층 corridor 가 전제다.

항목	목표	비고
최대 도메인	$\sim 8,000m$	OpenVDB + 계층 corridor 필수
단일 배관	< 1 초	corridor 한정 fine 탐색(95p)
전체(수백 배관)	< 1 분	평균 0.1~0.3 초/배관(병렬)
메모리	< 32GB	점유맵 + 탐색 자료구조

C++ 목표 처리량: 평면배열 + 정수 인덱스 + 힙 $\rightarrow$ 수백만~천만 확장/초(Python 대비 50~200 $\times$). 계층 corridor 가 확장 수 자체를 10~100 $\times$ 감축.

4. Phase 3 — C++ 엔진 + pybind11

Phase 2 명세를 C++20 으로 1:1 포팅한다. 골든셋을 expanded_nodes 까지 정확 재현하고, OpenVDB 희소 점유-계층 corridor-FCL 정밀 충돌.pybind11 바인딩·실행 CLI 를 추가한다. ctest 7 중 전부 통과.

4.1 빌드 체계

CMake + Visual Studio 17 2022 생성기, C++20. 한글 주석을 위해 MSVC /utf-8 필수. 코어는 외부 의존성 없이 빌드되고, OpenVDB/FCL/pybind11 은 옵션(vcpkg)으로 분리.

타겟 / 옵션	내용
routing3d_core	occupancy.cpp + sparse_occupancy.cpp + scene_io.cpp (정적 라이브러리).
routing3d_cli	실행 CLI(코어만). 항상 빌드.
-DUSE_OPENVDB=ON	routing3d_vdb 라이브러리(vcpkg OpenVDB).
-DUSE_FCL=ON	routing3d_fcl 라이브러리(vcpkg FCL).
-DBUILD_PYTHON_BINDINGS=ON	routing3d_cpp 파이썬 모듈(pybind11).

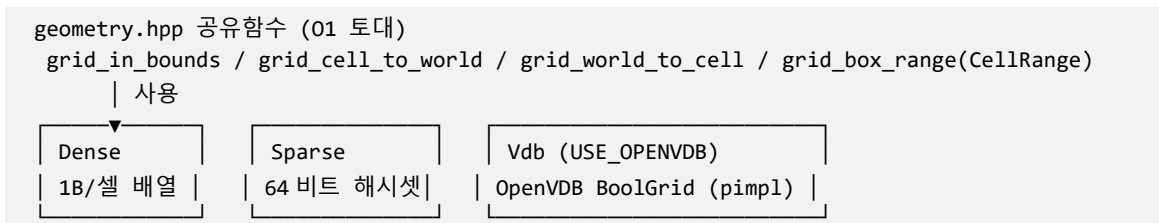
엔진(aster/cost/multi_route)은 점유 백엔드 무관 헤더 전용 템플릿이라 .cpp 가 없다(컴파일타임 다형성, 가상 디스패치 없음).

4.2 점유맵 백엔드 (geometry / occupancy / sparse / VDB)

개요

좌표/복셀화를 geometry.hpp 공유함수로 일원화해 백엔드 불문 동일 결과(불변식 O1)를 보장한다. 백엔드는 엔진 템플릿이 요구하는 최소 인터페이스(in_bounds/is_blocked/to_world/to_cell/lin/unlin/size/cell_mm/copy)를 제공한다.

흐름도



알고리즘 / 자료구조

- DenseOccupancy: 셀당 1 바이트 vector. lin() 선형 인덱스로 O(1) 질의.

- SparseOccupancy: std::unordered_set<uint64_t>. 패킹키
pack(c)=(i<42)|(j<21)|k(축당 21 비트). 메모리=O(점유 셀).
- VdbOccupancy: OpenVDB BoolGrid. add_box=grid->fill(bbox, true, true) 타일 최적화.
count=activeVoxelCount.
- add_box 는 grid_box_range(CellRange{lo,hi})로 셀 범위 산출 후 채움 → 세 백엔드 동일.

주요 함수 / 결과

함수	역할
in_bounds / is_blocked	격자 안/점유(밖=점유) 질의.
add_box / block_cell / count_blocked	복셀화 / 단일 점유 / 점유 셀 수.
to_world / to_cell / lin / unlin	좌표 변환 / 선형 인덱스 변환(엔진 g-closed 키).
copy()	독립 복사(다중배관 M2).

결과: 골든 01/02 가 Dense=Sparse=Vdb 동일 지표·경로. 8,000m³ 전체 채움(4.096×10^{15} 복셀) → OpenVDB 루트 타일로 1.2GB(압축비 >1000×). 단, 얇은 시트(한 축<노드 dim 128)는 leaf 타일까지만 → 전역 fine 격자는 비현실적이고 계층 corridor 가 필요(4.7).

실행명령어

```
# OpenVDB 백엔드 빌드(vcpkg)
cmake -S cpp -B cpp/build -G "Visual Studio 17 2022" -A x64 -DUSE_OPENVDB=ON ^
  -DCMAKE_TOOLCHAIN_FILE=D:/vcpkg/scripts/buildsystems/vcpkg.cmake -
  DVCPKG_TARGET_TRIPLET=x64-windows
ctest --test-dir cpp/build -C Release -R "occupancy|vdb" --output-on-failure
```

4.3 직교 A* (astar.hpp — 균일 + 비용함수)

개요

명세 §3/§4 를 헤더 전용 템플릿으로 구현. 점유 백엔드 무관(template<class Occ>).
결정성(A2/W1): (f, 삽입순서 counter) tie-break + 고정 이웃 순서로 Python 과 동일
경로·확장수 재현.

알고리즘 / 자료구조

- PQItem{f, counter, cell, dir} + PQ_cmp(최소 힙: 작은 f 우선, 동률은 작은 counter).
- 균일 astar: 상태=셀. closed=vector<uint8_t>(occ.size()), g/came=unordered_map.
h=manhattan×cell_mm.
- 비용 astar_weighted: 상태=(셀, 진입방향). 상태 인코딩 state = lin×7 + (dir+1), dir∈[-1,5].

- `closed=vector<uint8_t>(occ.size()×7)`. `CostModel<Occ>` 가 `move_cost/heuristic` 제공.
- 주의: `g/closed` 를 `lin()` 선형배열로 잡아 '경계 한정' 격자용. 초대형 전역은 해시 기반(`corridor.hpp`)으로 확장.

주요 함수 / 변수 (AStarResult)

이름	설명
<code>astar(occ, start, goal, step_cost=-1, max_expansions=-1)</code>	균일 비용. <code>step_cost<0 → cell_mm</code> .
<code>astar_weighted(occ, start, goal, params, max_expansions=-1)</code>	비용함수. 상태=(셀,방향).
<code>count_turns(path)</code>	방향 전환 수(인라인).
<code>AStarResult</code>	<code>success/path/length_mm/turns/expanded_nodes/cost_mm/elapsed_ms</code> .

결과

골든 01: expanded 22,856 / 02: 9,036 — Python 과 정확 일치. 동일 입력 → 동일 경로.

4.4 비용함수 (cost.hpp)

개요 / 알고리즘

`RouteParams` + `clearance_map`(BFS 거리변환) + `CostModel<Occ>` 를 헤더 템플릿으로 구현. 가산 페널티만 사용해 휴리스틱 admissibility 보존(C1).

- `clearance_map<Occ>(occ, R, connectivity)`: 셀별 장애물까지 거리(상한 R). 장애물=0, 멀수록 큼.
- `CostModel<Occ>::move_cost(to, prev_off, move_off) = cell_mm + (w_turn if 회전) + cell_penalty(to)`.
- `CostModel<Occ>::heuristic(c, goal) = manhattan × cell_mm`.
- `neighbors26()` 등 보조. 클리어런스 맵은 생성 시 1 회 사전계산.

주요 변수

`RouteParams{cell_mm, w_turn, w_clear, clearance_radius, clearance_connectivity, w_tier}`.
`baseline = {50, 500, 10, 2, 6, {}}` (Python 과 동일).

4.5 다중 배관 순차 라우팅 (multi_route.hpp)

개요 / 알고리즘

명세 §5 를 헤더 템플릿으로 구현. route_sequential 이 occ.copy() 사본에 배관을 누적하며 충돌 없이 배치. order_tasks 는 stable_sort(Python sorted 와 동일 결정성).

이름	역할
route_sequential<Occ>(occ, tasks, params, priority)	순차 라우팅 본체. MultiRouteResult<Occ> 반환.
order_tasks<Occ>(occ, tasks, priority)	우선순위 정렬(원본 불변).
snap_to_free_cell / mark_pipe	점유 시 빈 셀 스냅 / 경로(+반경) 점유표시.
PipeResult / MultiRouteResult<Occ>	배관 결과 / 전체 결과(occupancy 보관 + 지표).

결과

골든 03: 5/5 성공·충돌 0·총 28050mm. 계약 M1(충돌 0)·M2(원본 불변)
보존(DenseOccupancy::copy).

rip-up & reroute (Step 3.8)

순차 라우팅(greedy)은 먼저 깔린 배관이 통로를 선점하면 후순위 배관이 막힌다.
route_ripup 은 이를 '뜯어내고 다시 깔아' 해소한다. 무손실(채택 시 성공 +1) 결정적 알고리즘이라 성공 수가 절대 줄지 않고, Python-C++ 가 동일 결과를 낸다.

```
route_ripup(occ, tasks, params, priority, max_rounds, max_ripup)
0) route_sequential 베이스라인(placed = 성공 배관 경로 맵)
라운드 반복:
  실패 배관 f 마다:
    a) '장애물만' 점유맵에서 f 의 이상(ideal) 경로 - 실패하면 접근불가(영구 실패)
    b) ideal 이 가로지르는 기존 배관 = blocker (max_ripup 개 이하)
    c) blocker 뜯어냄 → f 배치 → blocker 전부 재라우팅
    d) f 성공 + blocker 전부 재배치 성공일 때만 채택(무손실), 아니면 폐기
  채택이 없으면 종료
```

- 단조성: 채택마다 +1 → 라운드·시도는 유한(초기 실패 수가 상한).
- 결정성: placed 를 std::map(키 정렬)으로 반복, blocker 는 인덱스 오름차순 → Python set/sorted 와 동일.
- CLI: routing3d_cli route --mode ripup. C ABI: r3d_route_ripup.

검증	결과
합성 혼잡(틈 2 개 벽)	순차 1/2 → rip-up 2/2 (긴 배관 1000→1300mm)

	우회, 짧은 배관 회생). +1 실증
project6 cell=100	순차 194/208 = rip-up 194/208 (Python·C++ 동일). 실패 14 건은 종단 PoC 접근성 한계로 무손실 rip-up 으로는 해소 불가
Python ↔ C++	동일 싼·동일 순서 → 동일 성공 수·총 길이(교차검증)

해석: rip-up 은 '혼잡(다른 배관이 막음)'은 해소하지만 '접근불가(종단이 장애물에 파묻힘)'는 구조상 해소할 수 없다. project6 의 잔여 실패는 후자 성격이며, 해소하려면 PoC 전처리/스냅 확장 또는 비용기반 negotiated-congestion(후속)이 필요하다.

4.6 scene.txt 입출력 (scene_io.hpp/.cpp)

개요 / 알고리즘

scene.txt v1 규격을 C++ 로 구현(Python↔C++ 공유 계약). 핵심은 Python repr(float)와 동일한 실수 표기를 C++ 로 재현하는 format_repr_double.

- format_repr_double: std::to_chars(scientific) → Python 임계값(decpt<=-4 또는 >16 → 지수)으로 재포맷(F4).
- 선택 문자열은 std::optional<std::string> 으로 WN(None) vs ""(빈) 구분(F3).
- 주의: std::from_chars(int)는 '+' 지수를 거부 → 지수 파싱 전 '+' 제거 필수(핵심 버그 수정).
- RouteTask 를 route_task.hpp 로 분리, utility_label() 은 None/빈을 ? 로(Python or 동일).

주요 함수 / 결과

함수	역할
dumps_scene / loads_scene	문자열 직렬화/역직렬화.
read_scene / write_scene	파일 입출력.
occupancy_from_doc	격자·장애물 → 점유맵.
format_repr_double	Python repr(float) 동일 표기.

결과: Python 픽스처(cpp/tests/fixtures/roundtrip.scene.txt)를 읽어 바이트 동일 재출력(F2). 픽스처는 LF 고정(.gitattributes), .gitignore 예외로 추적.

4.7 계층 corridor + 해시 A* (corridor.hpp)

개요

초대형(8,000m) 장면 대응. 현 astar 는 closed 를 occ.size() 배열로 잡아 거대 격자에서 할당 불가하다. 해시 기반 A* 와 coarse→fine corridor 로 탐색 공간을 제한한다.

흐름도 / 알고리즘

```
route_corridor(fine, coarse, start, goal, factor, radius)
1) coarse 점유맵에서 대략 경로(가이드) - astar_hashed(coarse, ...)
2) 가이드 경로를 반경 radius 로 팽창 → corridor(coarse 셀 키 집합)
3) fine A* - fine 셀의 coarse 셀이 corridor 에 있을 때만 탐색(tube 한정)
```

- $\text{pack20}(c) = (i < 40) | (j < 20) | k$ (축당 20 비트, 0..1,048,575). $8,000\text{m}/50\text{mm}=160,000 < 2^{20}$.
- `astar_hashed<Occ, InCorridor>`: `g/closed/came` 를 해시(pack20 키)로 → 메모리 $\propto$ 탐색한 셀 수.
- `in_corridor` 술어로 탐색 제한. 전체 허용(CorridorAll)이면 배열 astar 와 동일 경로/확장수.
- `CorridorRoute{fine, coarse_path, coarse_success, corridor_cells}`.

결과 / 한계

8,000m(160000^3) 로컬 배관 ~75ms, 벽 우회 corridor 가 전역 최단과 동일 길이·확장 55%.
한계: 해시 경로는 클리어런스(전역 거리변환 필요) 비활성, coarse 점유맵은 호출자가 구성.

4.8 FCL 정밀 충돌 (fcl_scene.hpp/.cpp)

개요 / 알고리즘

50mm 복셀 해상도를 넘는 sub-voxel 정밀 충돌 질의. 장애물 AABB 를 FCL dynamic-AABB-tree broadphase 에 등록하고, 배관을 반경 r 캡슐로 보고 정밀 검사한다(FCL/Eigen pimpl).

- `add_box` → `fcl::Boxd` 를 `DynamicAABBTreeCollisionManagerd` 에 등록.
- `collides_point` / `collides_sphere`: 점/구 충돌.
- `segment_clear`: 반경 r 캡슐(`Quaterniond::FromTwoVectors(UnitZ, dir)` 로 정렬) 비충돌 검사.
- `path_clear`: 경로 전 구간 캡슐 검사.

결과

FCL 점=AABB=점유맵 일치. 틸 200mm 에서 가는 파이프(r50) 통과·굵은 파이프(r150) 충돌(sub-voxel 판정). FCL 0.7.0 vcpkg(+eigen3/ccd/octomap).

4.9 pybind11 바인딩 (bindings.cpp)

개요 / 결과

C++ 엔진을 파이썬 모듈 routing3d_cpp 로 노출(pybind11 3.0.4, header-only, .venv pip 설치). 교차검증: 바인딩 경유 골든 01/02 경로셀까지 일치, 03-scene.txt 왕복 일치.

```
cmake -S cpp -B cpp/build -G "Visual Studio 17 2022" -A x64 -DBUILD_PYTHON_BINDINGS=ON ^
-Dpybind11_DIR=$(python -c "import pybind11;print(pybind11.get_cmake_dir())") ^
-DROUTING3D_TEST_PYTHON=<.venv\Scripts\python.exe>
ctest --test-dir cpp/build -C Release -R bindings --output-on-failure
```

4.10 실행 CLI (routing3d_cli.cpp + run.ps1)

개요

엔진을 명령줄에서 바로 실행하는 진입점. 외부 의존성 없이 코어만으로 빌드된다. scene.txt 를 읽어 라우팅하고 결과(경로 레이어 포함)를 scene.txt 로 저장하며, 입력 없이 내장 데모도 돌린다. 윈도우 한글 출력을 위해 chcp 65001.

명령	동작
demo [--out OUT]	내장 골든 03(5 배관 순차) 라우팅 + 요약.
route --in IN [--out OUT] [--mode multi single] [--priority longest]	scene.txt 라우팅 후 저장.
summary --in IN	scene.txt 요약.

결과 / 실행명령어

검증: demo 5/5 성공·총 28050mm(Python 일치). fixture 다중/단일=1/2 성공(장애물 내부 종단 1 건 정상 실패). 생성 scene.txt 를 summary 로 다시 읽어 동일(왕복 무손실).

```
.\run.ps1 # 내장 데모(자동 빌드+실행)
.\run.ps1 route --in scene.txt --out routed.scene.txt --mode multi
.\run.ps1 summary --in scene.txt
cpp\build\Release\routing3d_cli.exe demo # 빌드 후 직접 실행
```

4.11 Phase 3 결과 · 교차검증 요약

검증	결과
골든 01(빈 20³)	Dense=Sparse=Vdb 동일, length 2850mm·turns 2-expanded

	22,856(Python 일치)
골든 02(80 ³ 벽)	length 3950mm·turns 2-expanded 9,036(Python 일치)
골든 03(5 배관)	5/5 성공·충돌 0·총 28050mm(세 백엔드 동일)
scene.txt 왕복	Python 픽스처를 읽어 바이트 동일 재출력(F2)
8,000m 로컬 배관	해시 A* + corridor 로 ~75ms 라우팅(전역 최단 동일 길이)
FCL sub-voxel	틈 200mm: 가는 파이프(r50) 통과·굵은(r150) 충돌
pybind11	routing3d_cpp 경유 골든 01/02 경로셀·03-scene.txt 왕복 일치
rip-up(3.8)	합성 혼잡 1/2→2/2(+1) 실증, project6 는 베이스라인과 동일(접근불가 한계)
ctest 전체	8/8 통과(golden-scene_io-occupancy-corridor-vdb-fcl-bindings-capi)

5. C++ ↔ C# 인터롭 + HelixToolkit 뷰어

엔진은 C++ 로 두고 가시화를 C#(WPF + HelixToolkit)로 구현한다. C++ 코어를 의존성 없는 단일 DLL(C ABI)로 노출하고, C# 가 P/Invoke 로 호출해 라우팅 결과를 3D 로 그린다. 단계 P0(DLL) → P1(뷰어) → P2(인터랙티브) → P3a(뷰어 기능) → P3b(대형장면) → P3c(검증).

5.1 C ABI 네이티브 DLL (routing3d_capi)

개요 / 설계

C++ 코어만 링크한 외부 의존성 0 의 단일 DLL(약 261KB). 경계는 순수 C ABI(extern "C", cdecl): 불투명 핸들 + POD blittable 구조체 + 원시 배열. 예외는 절대 경계를 넘지 않게 try/catch→상태코드. 한글 이름을 위해 모든 문자열은 UTF-8.

- Level 1(문자열): r3d_route_scene_text — scene.txt in→out (mode multi|single).
- Level 2(핸들):
r3d_create/destroy/load_scene_text/set_grid/set_params/add_obstacle/add_task/set_task_endpoints/route_multi/route_task/route_corridor/route_ripup/get_result/copy_path/dump_scene_text.
- 불투명 핸들 R3dEngine = SceneDoc 보유. 콜리 할당 문자열은 r3d_free_string(동일 DLL malloc/free).
- 경로는 2 단계(R3dResult.path_len → r3d_copy_path, 콜리 할당).

주요 함수 / 결과

함수	역할
r3d_route_scene_text	Level 1: scene.txt 문자열을 받아 라우팅한 scene.txt 반환.
r3d_route_multi / route_task	전체 다중 / 단일 작업 라우팅(핸들).
r3d_route_corridor	대형 장면 계층 corridor(Sparse, 배열 미할당).
r3d_route_ripup	rip-up & reroute(Step 3.8).
r3d_get_result / r3d_copy_path	결과 지표 / 경로 셀 복사.

검증: ctest capi 가 DLL 경유 골든 03(5/5-28050mm) + 문자열 왕복 + 2000²×8 Sparse corridor 라우팅. ctest 8/8 통과.

5.2 C# WPF + HelixToolkit 뷰어 (Routing3D.Viewer)

개요 / 구성

.NET 9(net9.0-windows, x64 고정) WPF. P/Invoke + SafeHandle + 관리형 Engine 래퍼로 DLL 을 호출하고 HelixToolkit.Wpf 2.24.0(WPF 네이티브 MeshBuilder)로 3D 렌더. MVVM(SceneViewModel).

- Interop/: Native(P/Invoke, UTF-8 byte[] 마샬링) · R3dEngineHandle(SafeHandle) · Engine(OOP 래퍼).
- Model/: SceneData · SceneTextParser · UtilityColors(Python utility_colors 동일 규약) · CollisionFinder.
- ViewModels/SceneViewModel: 장애물 반투명 박스 머지 + 유틸리티별 경로 튜브 + 충돌 빨간 큐브.
- HelixToolkit 3.x 는 geometry 가 System.Numerics 로 분리돼 WPF 변환이 번거로워 2.24.0 채택.

단계별 기능

단계	내용
P1 뷰어	내장 데모/scene.txt 로드 → route_multi → 장애물·경로 3D 렌더.
P2 인터랙티브	작업 목록 선택 → 종단점(XYZ) 편집 → 단일/전체 재라우팅.
P3a 뷰어 기능	충돌 셀 시각화, 표시 토글(장애물/경로/충돌), 3D 클릭 종단점 지정(셀 스냅).
P3b 대형장면	corridor 라우팅 버튼(Sparse, OpenVDB 불필요 → DLL 코어 전용 유지).
P3c 검증	명령행 인자 scene.txt 로드 + --selftest 헤드리스(창 없이 전체 파이프라인) 검증.

5.3 실데이터 교차검증 (PostgreSQL → Python · C++ · C#)

기존 플랜트 DB(AUTOROUTINGV7 / TB_BIM_OBSTACLES)의 실제 장애물로 project6 씬을 만들고 세 구현이 같은 파일에서 같은 결과를 내는지 검증했다.

엔진	project6 (cell=100, 장애물 983·작업 208)
Python (route_sequential)	194/208 성공
C++ CLI (route_multi)	194/208 성공 · 총 3,400,800 mm

C# 뷰어 (--selftest, P/Invoke)	194/208 성공 · 총 3,400,800 mm
------------------------------	-----------------------------

→ Python 원형 → C++ 엔진 → C# 뷰어까지 완전 동일 결과. cell=200 동일 파일에서도 셋다 77/208·1,532,800mm 로 일치(격자 해상도에 따라 성공 수가 달라지므로 같은 파일끼리 비교).

실행명령어

```
# DLL 빌드 → 뷰어 빌드/실행
cmake --build cpp/build --config Release --target routing3d_capi
dotnet build csharp/Routing3D.Viewer.sln -c Release
dotnet run --project csharp/Routing3D.Viewer -c Release           # 창: 내장 데모
Routing3D.Viewer.exe path\to\scene.txt                          # 그 scene 로드
Routing3D.Viewer.exe --selftest scene.txt out.txt                # 헤드리스 검증
```

6. 전체 결과 요약 및 향후 과제

6.1 달성 요약

- Phase 1: Python 알고리즘(점유맵·A*·비용·다중배관·scene.txt·시각화·회귀 골든셋) 완료, pytest 203 통과.
- Phase 2: 알고리즘/포맷/골든셋/성능목표 명세 동결, 계약(불변식) 확정.
- Phase 3: C++ 엔진(코어 + OpenVDB + 계층 corridor + FCL + rip-up + pybind11 + CLI) 구현, ctest 8/8 통과.
- C# 인터롭: C ABI DLL + HelixToolkit 뷰어(P0~P3c). 실 DB 데이터로 Python=C++=C# 결과 일치 교차검증.
- 교차검증: 골든 01/02/03 을 expanded_nodes 까지 Python 과 정확 일치(세 백엔드 동일).

6.2 성능 목표 대비 현황

목표	현황
8,000m 도메인	OpenVDB 희소 + 계층 corridor 로 로컬 배관 ~75ms 실증.
단일 배관 < 1 초	corridor 한정 fine 탐색으로 달성 경로 확보(상세 벤치는 3.11).
메모리 < 32GB	8,000m³ 전체 채움 1.2GB(희소). 얇은 시트는 corridor 필요.
골든셋 재현	3 종 모두 허용범위 내(다수 항목 정확) 재현.

6.3 향후 과제 (Phase 3 잔여)

항목	내용
3.8 rip-up & reroute	구현·교차검증 완료(합성 +1 실증). 잔여: 접근불가(매장 PoC)는 PoC 전처리/스냅 확장, 혼잡은 비용기반 negotiated-congestion/CBS 로 추가 개선.
3.11 벤치 · 최적화	corridor 폭/해상도 튜닝, 클리어런스 로컬화, 라우팅에 FCL 통합, 독립 배관 병렬화.
3.12 회귀 리포트	표준 벤치 셋(골든 3 종 + project6 + 8,000m 합성)의 성능·정확도 리포트.
P3b' OpenVDB capi(선택)	VDB 백엔드를 C ABI 로 노출 + 런타임 DLL 동봉 — Sparse 로 충족돼 보류.

본 보고서는 2026-05-29 시점의 구현을 기준으로 한다. 최신 상태는 코드와 git 이력이
정답이다.